Git нұсқаулық

Бұл нұсқаулықты басынан аяғына дейін орындасаңыз, Git-ті орнатып, өз жобаңызды нұсқалармен басқарып, GitHub-қа жүктей аласыз. Әр бөлімде командалар мен түсіндірме бар, соңында тапсырмалар мен бекіту сұрақтары берілген.

1. Git деген не және не үшін керек

Git – файлдардағы өзгерістерді сақтап, бақылайтын **нұсқаларды басқару жүйесі** (Version Control System, VCS). Оны 2005 жылы Линус Торвальдс Linux ядросын әзірлеу үшін жасаған.

Git-сіз жұмыс былай көрінеді: `project_final.zip`, `project_final2.zip`, `project_final_ENDI_DURYS.zip`. Git бұл мәселені шешеді:

- **Тарих сақталады** – әр өзгерісті кім, қашан және не үшін жасағаны белгілі.
- **Кез келген нұсқаға қайту** – қате жіберсеңіз, бұрынғы жұмыс істеген күйге оралуға болады.
- **Параллель жұмыс** – бір жобада бірнеше адам бір-біріне кедергі жасамай жұмыс істейді.
- **Тәжірибе жасау қауіпсіз** – жаңа идеяны бөлек тармақта (branch) тексеріп, ұнамаса өшіріп тастайсыз.

Git пен GitHub – бір нәрсе емес

	Git	GitHub
Бұл не	Компьютерге орнатылатын бағдарлама	Интернеттегі сервис (сайт)
Қайда жұмыс істейді	Сіздің компьютеріңізде, интернетсіз де	Бұлтта, интернет арқылы
Негізгі міндеті	Нұсқаларды сақтау және басқару	Репозиторийлерді сақтау, бөлісу, командалық жұмыс
Баламалары	Mercurial, SVN	GitLab, Bitbucket

Қысқаша: **Git – құрал, GitHub – сол құралмен жасалған жобаларды сақтайтын орын.**

2. Git орнату

Windows

Алдымен Git бар-жоғын тексеріңіз. Кейбір компьютерлерде (мысалы, зертханада немесе VS Code орнатылғанда) Git бұрыннан орнатылған болуы мүмкін. `Win + R` басып, `cmd` жазыңыз (немесе Пуск мәзірінен PowerShell ашыңыз) да, мына команданы орындаңыз:

```
git --version
```

- `git version 2.x.x.windows.x` шықса — Git орнатылған, төмендегі орнату қадамдарын өткізіп жіберіп, 3-бөлімге өтіңіз.
- `'git' is not recognized as an internal or external command` шықса — Git жоқ, төмендегі қадамдармен орнатыңыз.

Орнату қадамдары:

1. git-scm.com/downloads сайтына кіріп, Windows нұсқасын жүктеңіз.
2. Орнатқышты іске қосыңыз. Көп параметрді әдепкі (default) күйінде қалдыруға болады.
3. «Choosing the default editor» қадамында ыңғайлы редакторды таңдаңыз (мысалы, Visual Studio Code).
4. «Adjusting the name of the initial branch» қадамында **Override…** → **main** таңдаңыз.
5. Орнатудан кейін **Git Bash** бағдарламасы пайда болады – командаларды осында жазамыз.

macOS

Терминалды ашып, мына команданы жазыңыз. Git жоқ болса, жүйе Xcode Command Line Tools орнатуды ұсынады:

```
git --version
```

Немесе Homebrew арқылы: `brew install git`.

Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install git
```

Орнатылғанын тексеру

```
git --version
```

Нәтижесінде `git version 2.x.x` сияқты жол шығуы керек. Шықса – Git дайын.

3. Алғашқы баптау (бір рет қана жасалады)

Git әр commit-ке авторды жазады, сондықтан алдымен өзіңізді таныстырыңыз. Аты мен email кейін GitHub-та тіркелген деректермен бірдей болғаны дұрыс.

```
git config --global user.name "Айгерім Серікова"
git config --global user.email "aigerim@example.com"
```

Негізгі тармақтың атын `main` деп қою (GitHub стандарты):

```
git config --global init.defaultBranch main
```

Редакторды таңдау (мысалы, VS Code):

```
git config --global core.editor "code --wait"
```

Барлық баптауды тексеру:

```
git config --list
```

`--global` жалаушасы баптау осы компьютердегі барлық жобаға қолданылатынын білдіреді. Оны жазбасаңыз, баптау тек ағымдағы репозиторийге ғана әсер етеді.

4. Негізгі ұғымдар

Git-ті түсінудің кілті – файлдың **үш күйін** білу. Әр өзгеріс жұмыс каталогынан staging аймағына, одан репозиторийге өтеді, ал `git push` оны GitHub-қа жібереді.



Git-тегі файл күйлері · 4 орын, 5 команда

Сол жақтағы көрсеткілер өзгерісті алға жылжытады, оң жақтағылар кері әкеледі. Алғашқы үшеуі интернетсіз жұмыс істейді.

Ұғым	Түсіндірме
Репозиторий (repository, repo)	Жоба файлдары мен оның бүкіл тарихы сақталатын орын
Жұмыс каталогы (working directory)	Компьютердегі файлдарды көріп, өңдейтін папка
Staging аймағы (index)	Келесі commit-ке кіретін өзгерістерді жинайтын «дайындық алаңы»
Commit	Жобаның белгілі бір сәттегі суреті (snapshot): автор, күн, хабарлама және бірегей хэш
Тармақ (branch)	Тарихтың тәуелсіз даму сызығы; әдепкісі – `main`

Ұғым	Түсіндірме
HEAD	Қазір қай commit пен тармақта тұрғаныңызды көрсететін нұсқағыш
Remote	Қашықтағы (мысалы, GitHub-тағы) репозиторий; әдетте `origin` деп аталады

Аналогия: staging – посылкаға заттарды салу, commit – қорапты жабып, жапсырма жабыстыру, push – пошта арқылы жіберу.

5. Бірінші репозиторий: қадамдық практика

Терминалды (Windows-та Git Bash) ашып, әр команданы ретімен орындаңыз.

1-қадам. Жоба папкасын жасау және оған кіру

```
mkdir my-first-repo
cd my-first-repo
```

2-қадам. Репозиторийді инициализациялау

```
git init
```

Папка ішінде жасырын `.git` папкасы пайда болады – бүкіл тарих осында сақталады. Оны қолмен өшірмеңіз.

3-қадам. Файл жасау және күйін тексеру

```
echo "# Менің бірінші жобам" > README.md
git status
```

`git status` файлды қызыл түспен **Untracked files** деп көрсетеді: Git оны көріп тұр, бірақ әлі бақыламайды.

4-қадам. Файлды staging аймағына қосу

```
git add README.md
git status
```

Енді файл жасыл түспен **Changes to be committed** бөлімінде тұр. Барлық өзгерген файлды бірден қосу үшін: `git add .`

5-қадам. Commit жасау

```
git commit -m "README файлын қосу"
```

`-m` кейін жазылатын мәтін – commit хабарламасы. Ол не өзгергенін қысқа түсіндіруі керек.

6-қадам. Файлды өзгертіп, тағы commit жасау

```
echo "Бұл жоба Git үйрену үшін жасалды." >> README.md
git add README.md
git commit -m "README-ге сипаттама қосу"
```

7-қадам. Тарихты көру

```
git log
git log --oneline
```

`git log` әр commit-тің хэшін (бірегей идентификатор, мысалы `a1b2c3d`), авторын, күнін және хабарламасын көрсетеді. `--oneline` әр commit-ті бір жолмен қысқа шығарады. Шығу үшін `q` пернесін басыңыз.

> ***Есте сақтаңыз:*** *негізгі цикл әрқашан бірдей –* ***өзгерту*** *→* ***`git add`*** *→* ***`git commit`****. Күмәндансаңыз,* `git status` *жазыңыз.*

6. .gitignore, diff және өзгерістерді болдырмау

.gitignore – Git елемейтін файлдар

Кейбір файлдарды репозиторийге салмау керек: құпия сөздер, уақытша файлдар, үлкен кітапханалар. Жоба түбірінде `.gitignore` файлын жасап, оларды тізіп жазыңыз:

```
# Құпия деректер
.env

# Тәуелділіктер
node_modules/
__pycache__/

# Жүйелік файлдар
.DS_Store
Thumbs.db

# Барлық .log файлдары
*.log
```

Дайын үлгілерді github.com/github/gitignore репозиторийінен алуға болады.

git diff – нақты не өзгерді

```
git diff            # әлі add жасалмаған өзгерістер
git diff --staged   # add жасалған, бірақ commit-ке түспеген өзгерістер
```

Қосылған жолдар `+` (жасыл), өшірілген жолдар `-` (қызыл) белгісімен көрсетіледі.

Өзгерістерді болдырмау

Жағдай	Команда	Не болады
Файлды өзгерттім, бірақ соңғы commit күйіне қайтарғым келеді	`git restore файл.txt`	Сақталмаған өзгерістер жойылады
Файлды `add` жасадым, staging-тен алғым келеді	`git restore --staged файл.txt`	Файл staging-тен шығады, өзгерістер қалады
Соңғы commit хабарламасын түзеткім келеді	`git commit --amend -m "Жаңа мәтін"`	Соңғы commit қайта жазылады

Жағдай	Команда	Не болады
Ескі commit-ті болдырмау	`git revert <хэш>`	Кері өзгеріс жасайтын жаңа commit құрылады

> ***Абайлаңыз:*** *`git restore` сақталмаған жұмысты қайтарылмастай өшіреді. `git commit --amend` командасын GitHub-қа жіберілмеген commit-ке ғана қолданыңыз. Командамен ортақ тарихты өзгерту үшін `git revert` қауіпсіз.*

7. Тармақтар (branch) және біріктіру (merge)

Тармақ – жобаның тәуелсіз даму сызығы. `main` тармағы әрқашан жұмыс істейтін нұсқа болып қалады, ал жаңа мүмкіндікті бөлек тармақта жасайсыз. Дайын болғанда оны `main`-ге біріктіресіз.

Негізгі командалар

```
git branch                  # тармақтар тізімі (* – ағымдағысы)
git branch feature-login    # жаңа тармақ жасау
git switch feature-login    # тармаққа ауысу
git switch -c feature-menu  # жасау + ауысу бір командада
git branch -d feature-login # біріктірілген тармақты өшіру
```

Ескі оқулықтарда `git checkout` командасын кездестіресіз. Ол да жұмыс істейді, бірақ тармақ ауыстыру үшін жаңа әрі түсінікті `git switch` ұсынылады.

Тармақпен жұмыс мысалы

```
git switch -c feature-about
echo "Біз туралы" > about.md
git add about.md
git commit -m "About бетін қосу"

git switch main
git merge feature-about
```

`main`-ге қайтқанда `about.md` жоғалады – бұл қалыпты, ол тек `feature-about` тармағында. `git merge` кейін ол `main`-де де пайда болады.

Конфликт (merge conflict) және оны шешу

Екі тармақта **бір файлдың бір жолы** әртүрлі өзгертілсе, Git қайсысын таңдарын білмейді. Ол файлды былай белгілейді:

```
<<<<<<< HEAD
Сәлем, әлем!
=======
Сәлеметсіз бе, әлем!
>>>>>>> feature-greeting
```

`<<<<<<< HEAD` пен `=======` арасы – ағымдағы тармақтағы нұсқа, `=======` пен `>>>>>>>` арасы – біріктірілетін тармақтағы нұсқа.

Шешу жолы:

1. Файлды редакторда ашыңыз.

2. Дұрыс нұсқаны қалдырыңыз (немесе екеуін біріктіріңіз).
3. `<<<<<<<`, `=======`, `>>>>>>>` белгілерін толық өшіріңіз.
4. Файлды сақтап, `git add файл.txt` жазыңыз.
5. `git commit` жасап, біріктіруді аяқтаңыз.

Біріктіруден бас тарту үшін: `git merge --abort`. Конфликт – қате емес, Git-тің сізден шешім сұрауы.

8. GitHub-пен жұмыс: remote, push, pull, clone

8.1. Аккаунт ашу

github.com сайтында тіркеліңіз. Email-ді 3-бөлімдегі `user.email` мәнімен бірдей көрсеткен дұрыс – сонда commit-тер профиліңізге байланады.

8.2. Компьютерді GitHub-қа қосу (SSH кілті)

GitHub құпия сөзбен push жасауға рұқсат бермейді. Ең ыңғайлы жол – SSH кілті.

1. Кілт жасаңыз (Enter-ді бірнеше рет басу жеткілікті):

   ```
   ssh-keygen -t ed25519 -C "aigerim@example.com"
   ```
2. Ашық кілтті көшіріңіз:

   ```
   cat ~/.ssh/id_ed25519.pub
   ```
3. GitHub-та **Settings → SSH and GPG keys → New SSH key** бетіне кіріп, көшірген мәтінді қойыңыз.
4. Байланысты тексеріңіз:

   ```
   ssh -T git@github.com
   ```
   `Hi <атыңыз>! You've successfully authenticated` деген жауап шықса – бәрі дұрыс.

Балама жол – HTTPS және **Personal Access Token** (Settings → Developer settings). Токенді құпия сөз сұралған жерге енгізесіз.

8.3. Жергілікті репозиторийді GitHub-қа жүктеу

1. GitHub-та **New repository** басып, атын `my-first-repo` деп қойыңыз. README қоспаңыз – бос репозиторий керек.
2. Терминалда жоба папкасында:

   ```
   git remote add origin git@github.com:<username>/my-first-repo.git
   git branch -M main
   git push -u origin main
   ```

`origin` – қашықтағы репозиторийдің шартты атауы. `-u` жалаушасы жергілікті `main`-ді GitHub-тағы `main`-ге байлайды, сондықтан кейін жай `git push` жазу жеткілікті.

8.4. Күнделікті жұмыс циклі

```
git pull          # GitHub-тағы жаңа өзгерістерді алу
# ... кодпен жұмыс ...
git add .
```

```
git commit -m "Не істегеніңізді жазыңыз"
git push          # өз өзгерістеріңізді GitHub-қа жіберу
```
Ереже: жұмысты бастамас бұрын `git pull`, аяқтағанда `git push`.

8.5. Біреудің жобасын көшіріп алу (clone)

```
git clone git@github.com:<username>/<repo>.git
cd <repo>
```
`git clone` бүкіл тарихымен бірге жобаны жүктейді және `origin`-ді автоматты түрде баптайды.

Пайдалы remote командалары

```
git remote -v        # байланысқан remote-тарды көру
git fetch            # өзгерістерді жүктеу, бірақ біріктірмеу
```
`git pull` = `git fetch` + `git merge`.

9. Командамен жұмыс: fork, pull request, жақсы commit

Fork және Pull Request (PR)

Fork – біреудің GitHub-тағы репозиторийінің өз аккаунтыңыздағы көшірмесі. **Pull Request** – «менің өзгерістерімді өз жобаңызға қосыңызшы» деген ұсыныс. Бұл ашық жобалар мен топтық тапсырмалардағы стандартты жол.

1. Жоба бетінде **Fork** түймесін басыңыз.
2. Өз fork-ыңызды `git clone` арқылы жүктеңіз.
3. Жаңа тармақ ашыңыз: `git switch -c fix-typo`.
4. Өзгерту → `git add` → `git commit`.
5. Тармақты жіберіңіз: `git push -u origin fix-typo`.
6. GitHub-та **Compare & pull request** түймесін басып, не өзгергенін сипаттаңыз.
7. Жоба иесі код шолуын (code review) жасап, PR-ды қабылдайды немесе түзету сұрайды.

Бір командадағы репозиторийде жұмыс істесеңіз, fork қажет емес: бірден тармақ ашып, PR жасайсыз.

Жақсы commit хабарламасы

Нашар	Жақсы
`update`	Тіркелу формасына `email` тексеруін қосу
`fix`	Себет бетіндегі баға есептеу қатесін түзету
`asdfgh`	`README`-ге орнату нұсқаулығын қосу
`көп өзгеріс`	Бөлек-бөлек бірнеше commit

Ережелер:

- Бір commit – бір логикалық өзгеріс.

- Хабарлама **не** істелгенін және қажет болса **неге** істелгенін айтады.
- Қысқа әрі нақты: бірінші жол 50 таңбадан аспағаны дұрыс.
- Жұмыс істемейтін кодты `main`-ге commit жасамаңыз.

10. Жиі кездесетін қателер және шешімдері

Қате хабарламасы	Себебі	Шешімі
`fatal: not a git repository`	Git репозиторийі жоқ папкадасыз	`cd` арқылы дұрыс папкаға кіріңіз немесе `git init` жасаңыз
`Please tell me who you are`	Аты мен email бапталмаған	3-бөлімдегі `git config` командаларын орындаңыз
`nothing to commit, working tree clean`	Жаңа өзгеріс жоқ	Қате емес – бәрі сақталған
`Changes not staged for commit`	Файлдар `add` жасалмаған	Алдымен `git add`, содан кейін `git commit`
`rejected ... (fetch first)` push кезінде	GitHub-та сізде жоқ commit-тер бар	`git pull` жасап, конфликт болса шешіп, қайта `git push`
`Permission denied (publickey)`	SSH кілті GitHub-қа қосылмаған	8.2-бөлімді қайталаңыз
`Authentication failed` (HTTPS)	Құпия сөз енгізілді	Құпия сөздің орнына Personal Access Token қолданыңыз
`CONFLICT (content): Merge conflict in ...`	Бір жол екі тармақта өзгерген	7-бөлімдегі конфликт шешу қадамдары
`git log`-тан шыға алмаймын	Терминал қарау режимінде	`q` пернесін басыңыз
Vim редакторы ашылып кетті	`-m` жазылмаған commit	`Esc`, кейін `:wq` жазып, Enter басыңыз

11. Командалар шпаргалкасы

Команда	Не істейді
`git config --global user.name "Аты"`	Автор атын баптау
`git init`	Жаңа репозиторий жасау
`git clone <url>`	Қашықтағы репозиторийді көшіру
`git status`	Файлдардың ағымдағы күйі

Команда	Не істейді
`git add <файл>` / `git add .`	Өзгерісті staging-ке қосу
`git commit -m "хабарлама"`	Snapshot (commit) жасау
`git log --oneline`	Commit тарихы
`git diff`	Сақталмаған өзгерістерді көру
`git restore <файл>`	Файлды соңғы commit күйіне қайтару
`git restore --staged <файл>`	Файлды staging-тен алу
`git revert <хэш>`	Commit-ті кері қайтаратын жаңа commit
`git branch`	Тармақтар тізімі
`git switch -c <ат>`	Жаңа тармақ жасап, ауысу
`git switch <ат>`	Тармаққа ауысу
`git merge <ат>`	Тармақты ағымдағыға біріктіру
`git remote add origin <url>`	Қашықтағы репозиторийді байлау
`git push`	Commit-терді GitHub-қа жіберу
`git pull`	GitHub-тан жаңартуларды алу
`git fetch`	Жаңартуларды біріктірмей жүктеу

12. Практикалық тапсырмалар

Әр тапсырманы орындағаннан кейін белгі қойыңыз.

1-тапсырма. Негіздер

- ☐ Git орнатып, `git --version` нәтижесін скриншот жасаңыз.
- ☐ Атыңыз бен email-ді баптаңыз.
- ☐ `about-me` репозиторийін жасап, ішіне `README.md` файлын (өзіңіз туралы 3 сөйлем) қосыңыз.
- ☐ Кемінде 3 commit жасаңыз, әрқайсысы мағыналы хабарламамен.
- ☐ `git log --oneline` нәтижесін көрсетіңіз.

2-тапсырма. .gitignore және болдырмау

- ☐ `.env` файлын жасап, `.gitignore`-ға қосыңыз. `git status` ол файлды көрсетпейтінін тексеріңіз.
- ☐ Бір файлды бұзып, `git restore` арқылы қалпына келтіріңіз.

3-тапсырма. Тармақтар және конфликт

- ☐ `feature-hobbies` тармағын ашып, `hobbies.md` файлын қосыңыз да, `main`-ге біріктіріңіз.
- ☐ Әдейі конфликт жасаңыз: `README.md` бірінші жолын `main`-де де, `feature-title` тармағында да әртүрлі өзгертіңіз.
- ☐ Конфликтті шешіп, біріктіруді аяқтаңыз.

4-тапсырма. GitHub

- ☐ SSH кілтін баптап, `ssh -T git@github.com` тексерісінен өтіңіз.
- ☐ `about-me` репозиторийін GitHub-қа push жасаңыз.
- ☐ GitHub сайтында README-ді өзгертіп, компьютерге `git pull` арқылы алыңыз.

5-тапсырма. Жұппен жұмыс

- ☐ Сыныптасыңыздың репозиторийін fork жасаңыз.
- ☐ Жаңа тармақта оның README-іне бір жол қосып, Pull Request ашыңыз.
- ☐ Сыныптасыңыздың PR-ын шолып, қабылдаңыз.

Тапсыру: GitHub репозиторийінің сілтемесі және ашылған PR сілтемесі.

13. Бекіту сұрақтары

A. Тест сұрақтары (бір дұрыс жауап)

1. Git дегеніміз не?
    - а) Код жазатын редактор
    - ә) Нұсқаларды басқару жүйесі
    - б) Бағдарламалау тілі
    - в) Бұлттық сақтау сервисі
2. Git пен GitHub-тың айырмашылығы қандай?
    - а) Ешқандай айырмашылық жоқ
    - ә) GitHub – Git-тің жаңа нұсқасы
    - б) Git – компьютердегі құрал, GitHub – репозиторийлерді сақтайтын сайт
    - в) Git тек Windows-та жұмыс істейді
3. Файлды staging аймағына қандай команда қосады?
    - а) `git commit`
    - ә) `git push`
    - б) `git add`
    - в) `git init`
4. `git status` командасы не көрсетеді?
    - а) Commit тарихын
    - ә) Файлдардың ағымдағы күйін
    - б) Тармақтар тізімін
    - в) Remote адрестерін
5. Жаңа тармақ жасап, бірден соған ауысатын команда:
    - а) `git branch -d new`
    - ә) `git merge new`

- б) `git switch -c new`
- в) `git log new`

6. `.gitignore` файлы не үшін керек?
   - а) Commit-терді өшіру үшін
   - ә) Git бақыламайтын файлдарды көрсету үшін
   - б) Құпия сөзді сақтау үшін
   - в) Тармақтарды жасыру үшін
7. `git pull` қандай екі команданың қосындысы?
   - а) `git add` + `git commit`
   - ә) `git fetch` + `git merge`
   - б) `git clone` + `git push`
   - в) `git init` + `git add`
8. GitHub-та PR (Pull Request) дегеніміз не?
   - а) Репозиторийді өшіру сұрауы
   - ә) Өзгерістерді жобаға қосу ұсынысы
   - б) Жаңа аккаунт ашу
   - в) Файлды жүктеп алу

Ә. Ашық сұрақтар

1. Git-тегі үш күйді (working directory, staging area, repository) өз сөзіңізбен түсіндіріңіз.
2. Неліктен `git add` пен `git commit` бөлек қадамдар? Бір командамен неге бірден сақтамаймыз?
3. Merge conflict қашан пайда болады және оны қалай шешесіз?
4. `git restore` пен `git revert` айырмашылығы неде? Қайсысы командамен ортақ тарих үшін қауіпсіз?
5. Жаңа мүмкіндікті `main` тармағында емес, бөлек тармақта жасаудың артықшылығы неде?
6. Сізге `rejected ... (fetch first)` қатесі шықты. Не болды және не істейсіз?
7. Жақсы commit хабарламасының 3 белгісін атаңыз және мысал келтіріңіз.